PLANT DISEASE DETECTION SYSTEM

PlantRx: Deep Learning-Based Leaf Disease Classification and Guidance

A PROJECT REPORT SUBMITTED TO

SRM INSTITUTE OF SCIENCE & TECHNOLOGY

IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE

AWARD OF THE DEGREE OF

MASTER OF SCIENCE IN

APPLIED DATA SCIENCE

BY

ABHIRUP URINDA (REG. NO. RA2532014010160)
SOURISH HALDER (RA2532014010161)
OM DESAI (RA2532014010138)
SANKET KURMI (RA2532014010156)

UNDER THE GUIDANCE OF

DR. S. SIVAKUMAR
Assistant Professor



DEPARTMENT OF COMPUTER APPLICATIONS

FACULTY OF SCIENCE AND HUMANITIES

SRM INSTITUTE OF SCIENCE & TECHNOLOGY

Kattankulathur – 603 203
Chennai, Tamil Nadu

OCTOBER - 2026

BONAFIDE CERTIFICATE

This is to certify that the project report titled “PLANT DISEASE DETECTION SYSTEM — PlantRx: Deep Learning-Based Leaf Disease Classification and Guidance” is a bonafide work carried out by Abhirup Urinda (Reg. No. RA2532014010160), Sourish Halder (RA2532014010161), Om Desai (RA2532014010138) and Sanket Kurmi (RA2532014010156) under my supervision for the award of the Degree of Master of Science in Applied Data Science. To my knowledge, the work reported herein is the original work done by these students.

_____

Dr. S. Sivakumar
Assistant Professor
Department of Computer Applications
(GUIDE)

_____

Dr. P. J. Arul Leena Rose
Professor & Head,
Department of Computer Applications

_____

INTERNAL EXAMINER

_____

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We express our sincere gratitude to everyone who supported and guided us throughout the completion of this project.

We thank our parents for their constant support, encouragement and motivation throughout our academic journey.

We sincerely thank Dr. T. R. Paarivendhar, Chancellor, SRM Institute of Science & Technology, for providing a supportive academic environment and the facilities required for our studies and project work.

We thank Dr. A. Duraisamy, Dean, Faculty of Science and Humanities, SRM Institute of Science & Technology, for his encouragement and support, and Dr. S. Albert Antony Raj, Deputy Dean, Faculty of Science and Humanities, for his valuable guidance and encouragement.

We are grateful to Dr. P. J. Arul Leena Rose, Professor & Head, Department of Computer Applications, for her support and guidance throughout the project.

We express our special thanks to our project guide, Dr. S. Sivakumar, Department of Computer Applications, for the valuable guidance, suggestions, support and encouragement provided during the review stages and the development of this project.

We also thank all the faculty members of the department for their suggestions during the project reviews, and our friends and everyone who supported us directly or indirectly in completing this project.

Abhirup Urinda (RA2532014010160)
Sourish Halder (RA2532014010161)
Om Desai (RA2532014010138)
Sanket Kurmi (RA2532014010156)

PROJECT COMPLETION CERTIFICATE

This is to certify that the project work titled “PLANT DISEASE DETECTION SYSTEM — PlantRx: Deep Learning-Based Leaf Disease Classification and Guidance” has been completed by the following students of the M.Sc. Applied Data Science programme, Department of Computer Applications, SRM Institute of Science and Technology, Kattankulathur, during the academic year 2026–2027, under the guidance of Dr. S. Sivakumar, Assistant Professor.

Abhirup Urinda (Reg. No. RA2532014010160)
Sourish Halder (Reg. No. RA2532014010161)
Om Desai (Reg. No. RA2532014010138)
Sanket Kurmi (Reg. No. RA2532014010156)

The students have completed the work as required by the department, and the project report has been prepared as per the departmental guidelines.

Date: _____

Place: Kattankulathur

_____
Dr. S. Sivakumar
Assistant Professor
(GUIDE)

_____
Dr. P. J. Arul Leena Rose
Professor & Head,
Department of Computer Applications

PLAGIARISM CERTIFICATE

This is to certify that the project report titled “PLANT DISEASE DETECTION SYSTEM — PlantRx: Deep Learning-Based Leaf Disease Classification and Guidance”, submitted by the students listed below to SRM Institute of Science and Technology in partial fulfilment of the requirements for the award of the degree of Master of Science in Applied Data Science, has been checked for plagiarism.

Abhirup Urinda (Reg. No. RA2532014010160)
Sourish Halder (Reg. No. RA2532014010161)
Om Desai (Reg. No. RA2532014010138)
Sanket Kurmi (Reg. No. RA2532014010156)

Plagiarism checking tool used: iThenticate

Similarity index obtained: 4 %

Date of checking: 12 October 2026

Dr. S. Sivakumar	**Dr. P. J. Arul Leena Rose**
Assistant Professor	Professor & Head,
(GUIDE)	Department of Computer Applications

The similarity report generated by iThenticate (Submission ID: trn:oid:::3117:637729231) is attached on the next two pages.

PlantRx_Capstone_Report_Final (1).pdf

Tripura University, Agartala

Document Details

Submission ID
trn:oid:::3117:637729231

Submission Date
Oct 12, 2026, 10:59 AM GMT+5:30

Download Date
Oct 12, 2026, 11:01 AM GMT+5:30

File Name
PlantRx_Capstone_Report_Final (1).pdf

File Size
1.6 MB

38 Pages

10,128 Words

53,934 Characters

4% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- **33** Not Cited or Quoted 4%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

2% Internet sources
3% Publications
0% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION....1
1.1 Background of the Study....1
1.2 Problem Statement....1
1.3 Objectives of the Project....2
1.4 Scope of the Project....2
1.5 Significance of the Project....2
1.6 Organization of the Report....3
CHAPTER 2 LITERATURE SURVEY AND EXISTING STUDIES....4
2.1 Review of Existing Systems / Research Works....4
2.2 Comparative Analysis....5
2.3 Research Gap / Limitations of Existing System....6
2.4 Proposed System / Proposed Approach....6
CHAPTER 3 SOFTWARE REQUIREMENT ANALYSIS....7
3.1 Hardware Specification....7
3.2 Software Specification....7
3.3 Software / Tools and Their Features....8
CHAPTER 4 SYSTEM ANALYSIS....9
4.1 Requirement Specification....9
4.2 Characteristics of Existing System....10
4.3 Proposed System / Proposed Approach....10
4.4 Feasibility Study....11
4.5 Software Requirement Specification....11
CHAPTER 5 SYSTEM DESIGN....12
5.1 System Architecture / Overall Workflow....12
5.2 Context Diagram....13
5.3 Use Case Diagram....13
5.4 Activity Diagram / State Diagram / UML Diagram....14
5.5 Data Flow Diagram / ER Diagram / Class Diagram / ML Workflow....15
5.6 Database Design / Dataset Design....16
5.7 User Interface Design....16
CHAPTER 6 SYSTEM IMPLEMENTATION....17
6.1 Modules Description....17
6.2 Dataset Description / Data Collection....17
6.3 Data Preprocessing....20
6.4 Feature Engineering / Feature Extraction....21
6.5 Model Architecture / Algorithm Design....21
6.6 System / Model Implementation....22
6.7 Validation Checks....24
6.8 User Interface Implementation....24
CHAPTER 7 TESTING AND EVALUATION....28

7.1 Test Cases....28
7.2 Unit Testing....28
7.3 Integration Testing....29
7.4 System Testing....29
7.5 Model Evaluation / Performance Evaluation....29
7.6 Performance Metrics....30
7.7 Comparative Analysis....30
CHAPTER 8 RESULTS AND CONCLUSION....31
8.1 Results....31
8.2 Discussion / Analysis....31
8.3 Limitations....32
8.4 Lessons Learnt....32
8.5 Conclusion....32
8.6 Future Enhancements....33
CHAPTER 9 APPENDICES....34
Appendix 1 – Plagiarism Certificate....34
Appendix 2 – Screenshots....34
Appendix 3 – Sample Coding....35
Appendix 4 – User Documentation....36
Appendix 5 – Glossary....37
Appendix 6 – Class Labels of the Dataset....37
CHAPTER 10 BIBLIOGRAPHY / REFERENCES....39

ABSTRACT

Plant diseases cause heavy yield and income losses every season. Checking leaves by eye is slow, depends on the person, and is often not possible in remote farming areas, so infections spread before anything is done. This project builds PlantRx, a system that identifies leaf diseases from a photograph and gives practical guidance on what to do next.

The system uses the PlantVillage cleaned dataset of 54,284 images in 38 classes across 14 crops, with 26 diseased and 12 healthy classes. The data were split 70:15:15 into 37,982 training, 8,126 validation and 8,176 test images with random seed 42. Analysis of the data showed uneven class sizes, a uniform image size of 256 × 256 pixels, and brightness that differs from crop to crop. These findings shaped the preprocessing. Class weighting and heavy augmentation were tried and dropped because they lowered accuracy.

Three ImageNet-pretrained models, MobileNetV2, DenseNet121 and EfficientNetB0, were fine-tuned and compared. MobileNetV2 reached about 97 percent test accuracy on all 38 classes and was chosen as the final model because it is light and fast. DenseNet121 reached about 94 percent. EfficientNetB0 was tried only on a smaller 22-class subset and gave about 85.76 percent, so it is not a direct comparison.

The chosen model was combined with a remedy database in a web application called PlantRx AI, built with a FastAPI back end and a React front end and packaged with Docker. A user uploads up to five leaf images, picks the plant and growth stage, and receives the predicted disease, a confidence score, the top three candidates, symptoms, treatment and prevention advice, and weather-based tips. An FAQ-style assistant answers follow-up questions. The result is a simple browser-based tool that shortens the time between the first symptoms and treatment.

LIST OF TABLES

Table 1.1 Project at a glance....3
Table 2.1 Qualitative comparison of diagnosis approaches....5
Table 2.2 Design characteristics of the three backbones....6
Table 3.1 Hardware specification....7
Table 3.2 Software specification....7
Table 3.3 Tools and their roles in the project....8
Table 4.1 Functional requirements....9
Table 4.2 Non-functional requirements....10
Table 4.3 Feasibility study....11
Table 5.1 External entities and interactions at the system boundary (textual context description)....13
Table 5.2 Use case specification derived from the Review I use case diagram....14
Table 5.3 Class responsibilities (from the Review I class diagram)....15
Table 5.4 Machine learning workflow....15
Table 5.5 Data entities described in the project documents....16
Table 6.1 Project modules and status....17
Table 6.2 Dataset summary and split....18
Table 6.3 Crop-wise image counts reported in Review I....18
Table 6.4 Model configuration details recorded in the project documents....21
Table 6.5 Inference pipeline for each uploaded image (Review II)....23
Table 6.6 Content returned by the inference service for each analysed image....23
Table 7.1 Functional test cases....28
Table 7.2 Model results....29
Table 7.3 Standard evaluation metrics for the 38-class classifier....30
Table 8.1 Objectives and outcomes....31
Table 9.1 Glossary of terms....37
Table 9.2 The 38 class labels of the PlantVillage cleaned dataset....37

LIST OF FIGURES

Fig. 5.1 End-to-end system and data-flow architecture (supplied in Review I: leaf image, preprocessing, trained CNN, prediction with confidence, remedy lookup, web display)..... 12

Fig. 5.2 Use case diagram of the Plant Disease Detection System (supplied in Review I)..... 13

Fig. 5.3 Class diagram of the system (supplied in Review I)..... 14

Fig. 6.1 Total images per crop species in the PlantVillage cleaned dataset (Review I)..... 19

Fig. 6.2 Interactive bar chart of the image count per class (exploratory analysis output)..... 19

Fig. 6.3 Brightness distribution per crop species, shown as box plots (exploratory analysis output)..... 20

Fig. 6.4 PlantRx AI home page..... 25

Fig. 6.5 Analyze Your Plant page for uploading up to five leaf images (JPG, PNG or WEBP, 10 MB each)..... 25

Fig. 6.6 Plant context information: species, growth stage and optional location..... 26

Fig. 6.7 Result page header: crop, predicted disease, confidence badge and model capability note..... 26

Fig. 6.8 Result page detail: observed symptoms, guidance plan, prediction confidence, top candidates and weather-aware advice..... 27

Fig. 9.1 How It Works section and AI Agricultural Assistant feature cards..... 34

Fig. 9.2 Total images per crop species (interactive chart)..... 34

Fig. 9.3 Distribution of image widths in the sampled images..... 35

Fig. 9.4 Image width against height by class in the sampled images..... 35

CHAPTER 1 INTRODUCTION

1.1 Background of the Study

Agriculture remains the primary source of livelihood for a large share of the rural population, and the health of a crop directly determines both its yield and the income of the farmer who grows it. Plant diseases are a persistent threat to this system. A leaf infection that is not recognised early can spread across a field within a growing season, reducing yield and increasing the cost of treatment. The Review I and Review II submissions of this project describe plant diseases as a major threat to food security that causes significant yield and economic losses each season.

Diagnosis in most farming communities still depends on visual inspection by the farmer or, where available, by an agricultural extension officer or plant pathologist. Visual diagnosis is slow and subjective, and it is often inaccessible in remote regions where trained experts are scarce. As a result, treatment may be delayed or applied on guesswork, which wastes time and inputs while the infection continues to spread.

Most leaf diseases produce visually distinctive symptoms such as lesions, discolouration, spotting, concentric rings or a mildew-like texture. These patterns make leaf disease a natural candidate for image classification. Convolutional Neural Networks (CNNs), combined with transfer learning from models pretrained on large image collections, can learn such patterns from labelled leaf photographs without the need for hand-designed features. A tool that diagnoses a disease from a photograph captured on an ordinary smartphone can therefore place a first level of diagnostic support directly in the field.

This report documents PlantRx, a leaf-disease detection and guidance system developed by a four-member team as an M.Sc. Applied Data Science project under the guidance of Dr. S. Sivakumar, Department of Computer Applications, SRM Institute of Science and Technology. The work was carried out in two review phases. Review I covered dataset acquisition, exploratory data analysis, preprocessing and transfer-learning model development. Review II covered the remedy database, the web application and its deployment packaging.

1.2 Problem Statement

Farmers and growers need a fast, accessible and consistent way to identify the disease affecting a crop leaf and to receive practical guidance on what to do next. Manual diagnosis is time-consuming, depends on scarce expertise and varies from person to person. The problem addressed by this project is to build an automated system that accepts a photograph of a plant leaf, classifies it into one of the supported crop–disease or healthy-leaf classes with a confidence score, and presents remedy-oriented guidance to the user in a browser-accessible interface.

1.3 Objectives of the Project

The objectives of the project, as set out in the Review I and Review II submissions, are:

- To acquire and profile the PlantVillage “cleaned” dataset and to analyse class balance, image resolution and brightness so that the findings can inform the preprocessing pipeline.
- To design an input pipeline that loads, resizes, normalises and batches leaf images consistently for training, validation and testing.
- To fine-tune and benchmark three ImageNet-pretrained CNN backbones (MobileNetV2, EfficientNetB0 and DenseNet121) and to select a final model for deployment.
- To build a remedy database that maps each of the 38 classes to a description, symptoms, treatment options and preventive practices.
- To integrate the selected model and the remedy database into a web application, PlantRx AI, that returns a diagnosis, a confidence score, the top candidate predictions and guidance within seconds.
- To provide supporting features around the diagnosis, namely multi-image upload, an FAQ-style assistant, user history, a PDF report export option and language selection, as described in the project documents.

1.4 Scope of the Project

The scope of the project covers image-based classification of leaf conditions for the 38 classes of the PlantVillage cleaned dataset, which span 14 crop species. The system takes leaf photographs as input and produces a predicted class and a confidence value. It supplies guidance retrieved from a prepared remedy database rather than generated by the model. The delivered application is a browser-based client–server system intended for on-demand, single-image inference on a CPU-only server.

The project does not claim to estimate disease severity or the affected leaf area. The result screen of the application states explicitly that these quantities are not provided by the current classification model. The system is also limited to the crop and disease classes present in the training dataset; leaves of other crops, or diseases outside the 38 classes, are outside the intended scope.

1.5 Significance of the Project

The significance of the project lies in shortening the interval between the first appearance of symptoms and the start of treatment. By presenting the diagnosis and the remedy on the same

page, the application reduces the dependence on an expert for routine, well-characterised leaf conditions. The use of a lightweight backbone allows the model to be served on modest hardware, which keeps deployment simple and inexpensive.

From an academic perspective, the project demonstrates a complete applied data science workflow: data acquisition, exploratory analysis, preprocessing, transfer learning, model comparison, packaging of the trained model, and integration into a usable application with a documented interface.

Table 1.1 Project at a glance

Item	Detail
Project title	Plant Disease Detection System
Subtitle	PlantRx: Deep Learning-Based Leaf Disease Classification and Guidance
Institution	SRM Institute of Science and Technology, Kattankulathur, Chennai
Department / Programme	Department of Computer Applications / M.Sc. Applied Data Science (2026–2027)
Project guide	Dr. S. Sivakumar
Team	Abhirup Urinda (RA2532014010160), Sourish Halder (RA2532014010161), Om Desai (RA2532014010138), Sanket Kurmi (RA2532014010156)
Dataset	PlantVillage cleaned dataset: 54,284 images, 38 classes, 14 crop species
Models studied	MobileNetV2, DenseNet121, EfficientNetB0 (ImageNet-pretrained, fine-tuned)
Final model	MobileNetV2

1.6 Organization of the Report

The report follows the chapter order given in the department handout. Chapter 2 looks at existing approaches and the gap this project fills. Chapters 3 and 4 list the hardware, software and requirements, and check whether the project is feasible. Chapter 5 explains the design, and Chapter 6 shows how the dataset, models and web application were built. Chapter 7 deals with testing and model evaluation. Chapter 8 gives the results, limitations and conclusion. Chapter 9 holds the appendices and Chapter 10 the references.

CHAPTER 2 LITERATURE SURVEY AND EXISTING STUDIES

2.1 Review of Existing Systems / Research Works

2.1.1 Manual and expert-based diagnosis

The conventional approach to leaf-disease diagnosis is visual inspection followed, where necessary, by laboratory confirmation. Visual inspection is quick when an experienced observer is available, but its accuracy depends on the observer's training, and different observers may disagree on visually similar conditions. Laboratory confirmation is more reliable but is slower and more expensive, and is not routinely accessible to small and remote farms. These limitations are the starting point for automated approaches.

2.1.2 Classical image processing and machine learning

Early automated systems extracted hand-designed features from leaf images, such as colour histograms, texture descriptors and shape measures, and passed them to classical classifiers. These methods can work on small, controlled datasets but depend heavily on the quality of the manual feature design. Their performance tends to fall when lighting, background or leaf orientation changes, because the engineered features do not generalise as well as features learned from data.

2.1.3 Convolutional neural networks and transfer learning

Deep CNNs learn hierarchical visual features directly from labelled images, which removes the need for manual feature engineering. Work on image-based plant disease detection using deep learning on the PlantVillage image collection established the dataset as a widely used benchmark for this task. Deep residual networks and later architectures made it practical to train very deep models, and models pretrained on large general image collections can be adapted to a new domain with comparatively little data.

Transfer learning is the technique of initialising a network with weights learned on a large source task, here ImageNet classification, and then fine-tuning some or all layers on the target task. As noted in the Review I report, this converges faster and generalises better than training from scratch on a dataset of the size used in this project. The PlantRx project adopts this approach for all three of its candidate backbones.

2.1.4 The PlantVillage dataset

The PlantVillage collection is an open image repository of healthy and diseased crop leaves. The project uses a "cleaned" version of this dataset that contains 54,284 images grouped into 38 classes across 14 crop species. Because the images are widely used, results obtained on this dataset are easy to compare in principle, but they should be interpreted with the

understanding that a held-out split of the same collection does not by itself demonstrate performance on photographs taken in uncontrolled field conditions.

2.1.5 Backbone architectures considered

Three architectures were selected for comparison. **MobileNetV2** is a lightweight network built around inverted residual blocks with linear bottlenecks, designed for efficient inference on limited hardware. **DenseNet121** connects each layer to all preceding layers within a dense block so that feature maps are reused; this improves gradient flow and feature propagation at the cost of more parameters than MobileNetV2. **EfficientNetB0** is the baseline model of a family obtained by compound scaling of network depth, width and input resolution.

2.2 Comparative Analysis

Table 2.1 compares the broad families of approaches discussed above using qualitative criteria only. No numerical performance claims are made for approaches that were not implemented in this project. Table 2.2 summarises the design characteristics of the three backbones used in the project.

Table 2.1 Qualitative comparison of diagnosis approaches

Approach	Strengths	Limitations	Relevance to PlantRx
Manual visual inspection	No equipment; immediate	Subjective; needs expertise; slow to scale	Problem context
Laboratory diagnosis	High reliability	Costly; delayed results; limited access	Reference standard, not a field tool
Hand-crafted features + classical ML	Light computation; interpretable features	Depends on manual feature design; sensitive to imaging conditions	Superseded by learned features
CNN trained from scratch	Learns task-specific features	Needs large data and long training	Not adopted
CNN with transfer learning	Faster convergence; strong features from pretraining	Depends on dataset coverage and domain match	**Adopted**

Table 2.2 Design characteristics of the three backbones

Backbone	Key design idea	Practical implication for the project
MobileNetV2	Inverted residual blocks with linear bottlenecks; depthwise separable convolutions	Small and fast; loads quickly and predicts on a CPU-only server
DenseNet121	Dense connectivity: each layer receives feature maps of all earlier layers in its block	More parameters; improved feature reuse; heavier to serve
EfficientNetB0	Compound scaling of depth, width and resolution	Evaluated only in a reduced preliminary run in this project

2.3 Research Gap / Limitations of Existing System

Many published studies stop at reporting classification accuracy on a held-out split and do not describe the path from a trained model to a tool that a farmer can actually use. The gap addressed by this project is integration. The project connects the trained classifier to a remedy database, a browser interface, a follow-up question facility and a history view, so that the diagnosis is accompanied by actionable guidance.

Existing tools for plant disease identification also vary in how clearly they communicate the reliability of a prediction. In PlantRx the confidence score and the three most likely classes are displayed, and the homepage describes a low-confidence flag for results below 60 percent so that treatment is not applied on an uncertain prediction. In addition, the application states plainly which quantities it does not provide, namely severity and affected area.

2.4 Proposed System / Proposed Approach

The proposed approach is a transfer-learning classifier wrapped in a client–server web application. The classifier is trained on the PlantVillage cleaned dataset using a 70:15:15 split. Three pretrained backbones are fine-tuned under matched conditions and compared on the held-out test data, and the best-suited model is saved with its class-index mapping. The web back end loads the model once at start-up, validates each uploaded image, preprocesses it in the same way as during training, predicts, looks up the remedy and returns the result as structured data to the front end. The detailed design is given in Chapter 5 and the implementation in Chapter 6.

CHAPTER 3 SOFTWARE REQUIREMENT ANALYSIS

3.1 Hardware Specification

The hardware requirements differ between the training phase and the deployment phase. Training of the deep models needs GPU acceleration, which in this project was provided by the cloud notebook environment. Deployment is a single-image, on-demand inference task, so a CPU-only server is sufficient, and the end user needs only a device with a web browser and the ability to upload or capture an image.

Table 3.1 Hardware specification

Phase	Hardware requirement	Source / remark
Training	GPU runtime (NVIDIA T4 via Google Colab); 12–16 GB RAM; 5–8 GB storage	Review I, Section 3.4; Review II, Section 3.6
Deployment (server)	CPU-only inference is sufficient; container-capable host	Review I and Review II
Client	Desktop, laptop or smartphone with a browser; optional camera for capture	Review II, Section 4

3.2 Software Specification

Table 3.2 Software specification

Category	Specification
Training environment	Python 3.x, TensorFlow/Keras, NumPy, Pandas, Matplotlib/Plotly, scikit-learn; Google Colab with NVIDIA T4 GPU
Pretrained models	MobileNetV2, EfficientNetB0, DenseNet121 (ImageNet weights, fine-tuned)
Image handling	Pillow, NumPy
Back end	Python with FastAPI, served by Uvicorn; Pydantic for request and response validation
Front end	JavaScript (React with Vite), managed with npm / Node.js
Remedy database	JSON / SQLite lookup
Configuration	python-dotenv and .env environment variables
Packaging	Docker (Dockerfile and docker-compose.yml)

3.3 Software / Tools and Their Features

Table 3.3 Tools and their roles in the project

Tool	Role and features
Google Colab	Hosted notebook environment with a T4 GPU, used for dataset exploration, preprocessing and model training. Three implementation notebooks are referenced in Appendix 3.
TensorFlow / Keras	Deep learning framework used to load ImageNet-pretrained backbones, fine-tune them and save the trained model together with its weights.
NumPy / Pandas	Array handling and tabular summaries of image counts and image statistics.
Matplotlib / Plotly	Static and interactive charts for class balance, resolution and brightness analysis.
scikit-learn	Support library for evaluation utilities.
Pillow	Opening, validating and converting uploaded images to RGB before resizing.
FastAPI / Uvicorn	REST API that receives uploads, runs the inference pipeline, queries the remedy database and returns JSON.
Pydantic	Validation of request and response data.
React / Vite	Front end providing the upload interface with previews, the results panel, history page and assistant window.
Docker	Packaging so that the back end and front end start in the same way on any machine.

CHAPTER 4 SYSTEM ANALYSIS

4.1 Requirement Specification

4.1.1 Functional requirements

The functional requirements below come from the project plan, the Review II description of the application and the interface screens.

Table 4.1 Functional requirements

ID	Requirement	Where it appears
FR1	Accept leaf images by drag and drop, file browsing or device camera, up to five images (JPG, PNG or WEBP, 10 MB each), with a preview before submission	Upload screen (Fig. 6.5)
FR2	Let the user select the plant species, growth stage and an optional location or farm name	Plant context screen (Fig. 6.6)
FR3	Preprocess each image in the same way as in training and predict one of 38 classes	Section 6.6.2
FR4	Show the disease name, confidence score and the three most likely classes	Result screens (Figs. 6.7, 6.8)
FR5	Show observed symptoms, immediate actions, treatment options and prevention guidance from the remedy database	Result screen (Fig. 6.8)
FR6	Show weather-aware advice (temperature, humidity, rain risk)	Result screen (Fig. 6.8)
FR7	Provide an FAQ-style assistant for follow-up questions	Section 6.6.5
FR8	Provide sign-in and a per-user history of diagnoses	Section 5.6.3
FR9	Offer a PDF report download and language selection	Result screen (Fig. 6.7)
FR10	Flag low-confidence results (below 60 percent)	Feature cards (Fig. 9.1)

4.1.2 Non-functional requirements

Table 4.2 Non-functional requirements

Attribute	Requirement
Performance	A single uploaded image should return a result within a few seconds on a CPU-only server (Review II, Section 4.1).
Usability	The application should run in a standard browser without installation and present the diagnosis and the remedy on the same page.
Portability	The back end and front end should start together in the same way on any machine through Docker.
Maintainability	The remedy text should be editable without changing model or application code.
Reliability	The system should handle unsupported or unreadable files gracefully, and should communicate low confidence rather than present an uncertain result as definite.
Security and privacy	Secret keys and settings are kept in environment variables rather than source code. Each user should see only their own history.
Transparency	The interface should state what the model does not provide (severity and affected area).

4.1.3 User characteristics

The main users are farmers and growers who may have little technical training. They need a screen that is easy to read, works on a phone, and gives an answer without extra steps. A second group is agricultural students and extension workers, who can use the confidence score and the list of top predictions to judge how reliable a result is. Both groups need the advice to be short, practical and written in plain language.

4.2 Characteristics of Existing System

The existing process for the target users is manual. A farmer observes the symptoms, compares them with experience or with printed material, and may consult an extension officer or expert. The process has no recorded history, gives no quantified indication of how certain the identification is, and leaves the choice of treatment to the individual. Generic online image-identification tools do not necessarily cover the crop and disease classes relevant to the user, and they seldom connect the identification to a practical treatment plan or to local weather conditions.

4.3 Proposed System / Proposed Approach

The proposed system, PlantRx AI (Plant Health Assistant), accepts up to five leaf images, passes each through a trained CNN classifier and returns the predicted disease, the confidence and the top three candidate predictions. The result is accompanied by guidance from the remedy database, a weather-aware advice panel, and an assistant for follow-up questions. A signed-in

user can return to previous results through a history page. Chapter 5 describes the architecture in detail.

4.4 Feasibility Study

Table 4.3 Feasibility study

Dimension	Assessment
Technical	The PlantVillage dataset, ImageNet-pretrained backbones and the TensorFlow/Keras framework are freely available. Training was feasible on a cloud GPU runtime, and MobileNetV2 is light enough to serve on a CPU-only host.
Operational	The system runs in a browser, requires no installation by the user, and delivers diagnosis and guidance in one step, which suits non-expert users.
Economic	The project relies on open datasets, open-source libraries and a free-tier cloud notebook for training. Deployment needs only a CPU server. No licence costs are recorded in the supplied material.
Schedule	The work was organised in two reviews: model development in Review I and application development in Review II.
Risk	Model performance on field photographs may be lower than on the dataset; this is addressed in the limitations (Section 8.3).

4.5 Software Requirement Specification

Purpose. To specify the behaviour of PlantRx AI, a web application for classifying leaf images into 38 crop–disease or healthy classes and presenting guidance. **Intended users.** Farmers, growers, agricultural students and extension workers. **Operating environment.** A modern browser on a desktop or mobile device, with a server that can run a Python back end and load a TensorFlow/Keras model on CPU. **Constraints.** The system covers only the 38 PlantVillage classes; it must reject unreadable or unsupported files; and it should present confidence with every prediction. **Assumptions.** The uploaded image is a reasonably clear photograph of a single leaf; the application framing check shown in the interface (Fig. 6.9) is an aid and not a guarantee of suitability.

CHAPTER 5 SYSTEM DESIGN

5.1 System Architecture / Overall Workflow

PlantRx AI follows a client–server design. The front end runs in the user's browser and handles sign-in pages, the upload interface with previews, the results panel, the history page and the assistant window. The back end is a Python FastAPI service that receives the uploaded image through a REST API, runs the inference pipeline, queries the remedy database and returns the result as JSON. The trained model is loaded once at server start-up. The overall data flow defined in Review I is shown in Fig. 5.1.

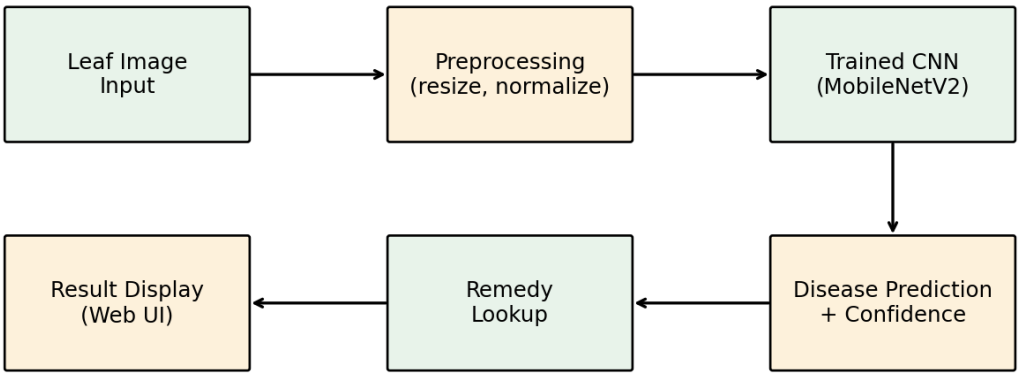


Fig. 5.1 End-to-end system and data-flow architecture (supplied in Review I: leaf image, preprocessing, trained CNN, prediction with confidence, remedy lookup, web display)

The end-to-end user journey described in Review II is: the user signs in and opens the website; uploads a leaf photograph; the server preprocesses the image and the model predicts the disease; the remedy is retrieved from the database; and the diagnosis, confidence and remedy are displayed and saved to the user's history. The user may then ask the assistant further questions or open the history page.

5.1.1 Layers of the system

The system can be seen as four layers. The **presentation layer** is the React front end, where the user uploads images, enters plant details and reads the result. The **application layer** is the FastAPI service, which validates the request, runs the inference pipeline and builds the response. The **model layer** holds the saved network and the class-index mapping. The **data layer** holds the remedy database and, for signed-in users, the history records. Keeping these layers apart means that the model can be replaced, or the remedy text edited, without changing the rest of the system.

5.2 Context Diagram

A formal context diagram was not among the supplied project artefacts, and one has not been drawn here so that no unsupplied diagram is presented as part of the original project material. The context of the system is therefore summarised in Table 5.1 in textual form. At the context level the system is a single process, PlantRx AI, that exchanges data with the external entities listed.

Table 5.1 External entities and interactions at the system boundary (textual context description)

External entity	Data sent to PlantRx AI	Data received from PlantRx AI
User (farmer / grower)	Leaf images, plant species and growth stage, optional location, questions, language choice	Predicted disease, confidence, top candidates, symptoms, guidance plan, weather advice, history, PDF report option
Remedy database	Query by predicted class label	Description, symptoms, treatment and prevention text
Trained model artefact	Preprocessed image batch	Class probabilities for 38 classes
Weather information	Farm location	Temperature, humidity and rain-risk values shown in the interface

5.3 Use Case Diagram

The use case diagram supplied in Review I (Fig. 5.2) shows a single actor, the User, who uploads a leaf image, has it preprocessed and validated, obtains a disease classification from the CNN model, views the prediction and confidence score, and views a remedy suggestion and disease information. The use cases of preprocessing and validation, and of browsing disease information, are related to the primary use cases by include or extend relationships.

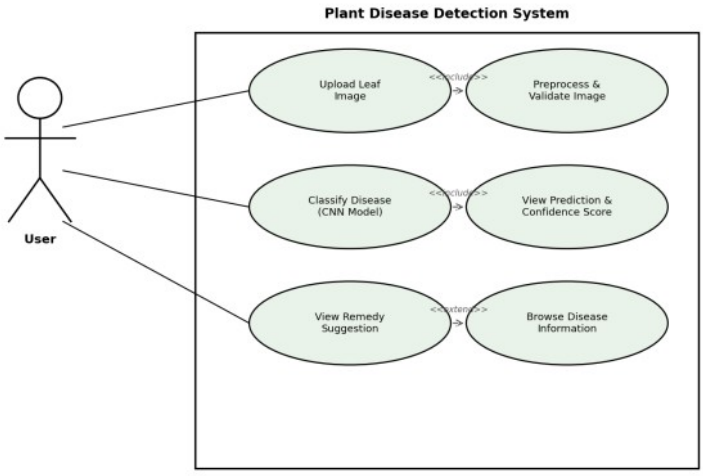


Fig. 5.2 Use case diagram of the Plant Disease Detection System (supplied in Review I)

Table 5.2 Use case specification derived from the Review I use case diagram

Use case	Actor / trigger	Description and outcome
Upload leaf image	User selects or captures a leaf photograph	The image is added to the selection and a preview is shown
Preprocess and validate image	Included by the upload and classification use cases	File type and readability are checked; the image is converted, resized and normalised
Classify disease (CNN model)	User presses the analyse button	The trained CNN returns a probability for each of the 38 classes
View prediction and confidence score	Classification complete	The predicted disease, confidence and top candidates are displayed
View remedy suggestion	Prediction available	Symptoms, immediate actions, treatment options and prevention guidance are shown
Browse disease information	Optional, from the result or assistant	The user reads further information about the identified disease

5.4 Activity Diagram / State Diagram / UML Diagram

The class diagram supplied in Review I (Fig. 5.3) models the core components of the system. Review II states that this structure was carried over to the implementation, with each class realised as a separate function or module. The principal relationships are that the User uses the WebApplication; the ImageUploader feeds the ImagePreprocessor, which feeds the CNNClassifier; the CNNClassifier produces a DiseaseResult; and the CNNClassifier queries the RemedyDatabase.

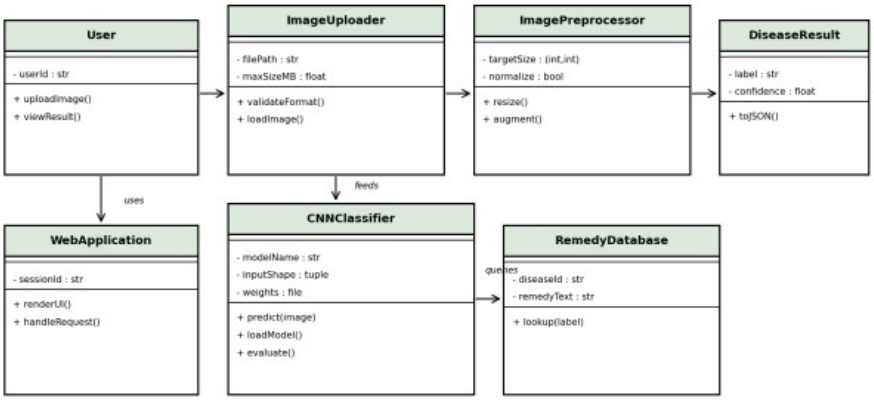


Fig. 5.3 Class diagram of the system (supplied in Review I)

No separate activity or state diagram was supplied. The behaviour of the system over time is described by the end-to-end workflow in Section 5.1 and by the six-step inference pipeline in Table 6.5.

5.5 Data Flow Diagram / ER Diagram / Class Diagram / ML Workflow

The data flow of the application is the one shown in Fig. 5.1. The responsibilities of the classes in Fig. 5.3 are summarised in Table 5.3. The machine learning workflow, which is the central technical contribution of the project, is summarised in Table 5.4.

Table 5.3 Class responsibilities (from the Review I class diagram)

Class	Attributes	Operations
User	userId	uploadImage(), viewResult()
WebApplication	sessionId	renderUI(), handleRequest()
ImageUploader	filePath, maxSizeMB	validateFormat(), loadImage()
ImagePreprocessor	targetSize, normalize	resize(), augment()
CNNClassifier	modelName, inputShape, weights	predict(image), loadModel(), evaluate()
DiseaseResult	label, confidence	toJSON()
RemedyDatabase	diseaseId, remedyText	lookup(label)

Table 5.4 Machine learning workflow

Step	Stage	Description
1	Data acquisition	PlantVillage cleaned dataset (54,284 images, 38 classes, 14 crops) obtained and split 70:15:15 with random seed 42
2	Exploratory analysis	Class balance, image resolution and brightness profiled across crops and classes
3	Preprocessing	Images loaded, resized to the model input resolution, normalised and batched
4	Model development	Three ImageNet-pretrained backbones fine-tuned end to end at a low learning rate under matched conditions
5	Evaluation	Models compared on the held-out test data; a final backbone selected
6	Packaging	Selected model saved with trained weights and class-index mapping for single load at server start-up
7	Serving	Inference pipeline exposed through the REST API with remedy lookup

5.6 Database Design / Dataset Design

5.6.1 Dataset design

The dataset is organised as a folder per class, with 38 class folders named in the form Crop_*Condition, for example Tomato*_Early_blight or Apple___healthy. The split into training (37,982 images), validation (8,126 images) and test (8,176 images) sets follows a 70:15:15 ratio with random seed 42. The details are given in Section 6.2.

5.6.2 Remedy database design

The remedy database is a structured lookup that maps each of the 38 class labels to its disease description, symptoms, recommended treatment and preventive measures. Healthy-leaf classes map to general care advice. It is stored in a JSON or SQLite format so that no separate database server is needed and the text can be edited without touching the model or the application code.

5.6.3 History record design

Review II describes a per-user history in which each entry records the uploaded leaf image, the predicted disease, the confidence score, the suggested remedy and the date and time of the analysis. Access to the history requires sign-in, and each user can see only their own entries.

Table 5.5 Data entities described in the project documents

Entity	Fields	Source
Remedy entry	Class label, disease description, symptoms, treatment options, preventive measures	Review II, Section 3.3
History entry	Leaf image, predicted disease, confidence, remedy, date and time	Review II, Section 4.4
User account	Sign-in credentials managed by the application	Review II, Section 4.4

5.7 User Interface Design

The interface uses a dark theme with green accents and a top navigation bar with Home, Analyze Leaf, History and AI Assistant, a language selector and a user icon. The design follows the three steps shown on the home page: upload, analyse and get guidance. The main screens are shown in Section 6.8.

Three ideas guide the design: a single clear path from upload to guidance, plant details collected before the analysis, and a result page that shows the confidence, the top three candidates and the limits of the model next to the advice.

CHAPTER 6 SYSTEM IMPLEMENTATION

6.1 Modules Description

The project is organised into six modules. The first four were complete at Review I and the remaining two were scoped for Review II and described in the Review II report.

Table 6.1 Project modules and status

No.	Module	Description	Reported stage
1	Data acquisition and EDA	PlantVillage cleaned dataset sourced and split 70:15:15; class balance, resolution and brightness analysed	Review I
2	Preprocessing pipeline	Input pipeline for loading, resizing, normalising and batching images	Review I
3	Model development and training	Three ImageNet-pretrained backbones fine-tuned end to end at a low learning rate	Review I
4	Model evaluation and selection	Models evaluated on the held-out test set and compared	Review I
5	Remedy database	Lookup mapping 38 classes to remedy and prevention guidance	Review II
6	Web application and deployment	Upload, prediction, remedy display, assistant and history in PlantRx AI	Review II

6.2 Dataset Description / Data Collection

6.2.1 Source and composition

The project uses the PlantVillage cleaned dataset, shared through a project Google Drive link listed in the references. The dataset contains 54,284 leaf images in 38 classes across 14 crop species. Of the 38 classes, 26 represent an active disease, for example bacterial spot, early blight, leaf mould or citrus greening, and 12 represent a healthy-leaf baseline for their respective crop. Tomato is the most represented crop, contributing 10 of the 38 classes and 18,160 images, while several crops (Orange, Soybean, Squash, Blueberry and Raspberry) are represented by a single class each.

Table 6.2 Dataset summary and split

Attribute	Value
Dataset	PlantVillage cleaned dataset
Total images	54,284
Classes / crop species	38 / 14 (26 diseased classes, 12 healthy classes)
Training set	37,982 images
Validation set	8,126 images
Test set	8,176 images
Split ratio / random seed	70:15:15 / 42
Source image size	256 × 256 pixels (all sampled images)
Model input size	224 × 224 pixels for the relevant model workflow (128 pixels in the reduced EfficientNetB0 run)

The split counts sum to 54,284 (37,982 + 8,126 + 8,176), which agrees with the reported total.

6.2.2 Crop-wise composition

Table 6.3 Crop-wise image counts reported in Review I

Crop	Classes	Images	Crop	Classes	Images
Tomato	10	18,160	Pepper, bell	2	2,475
Orange	1	5,507	Potato	3	2,152
Soybean	1	5,090	Cherry	2	1,906
Grape	4	4,062	Squash	1	1,835
Corn (maize)	4	3,852	Strawberry	2	1,565
Apple	4	3,171	Blueberry	1	1,502
Peach	2	2,657	Raspberry	1	371

6.2.3 Class balance

The dataset is heavily imbalanced. Tomato alone accounts for about one third of all images, while Raspberry contributes only 371, a ratio of roughly 49 to 1 between the largest and smallest crop groups (Fig. 6.1). The pattern persists at the class level: the largest classes exceed 2,000 images, and the smallest, for example Potato healthy, has 152 images (Fig. 6.2).

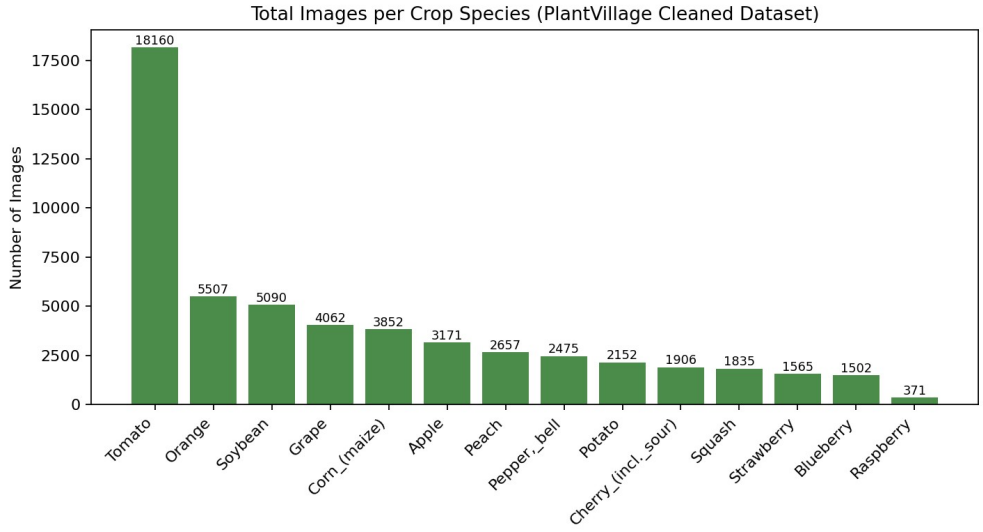


Fig. 6.1 Total images per crop species in the PlantVillage cleaned dataset (Review I)

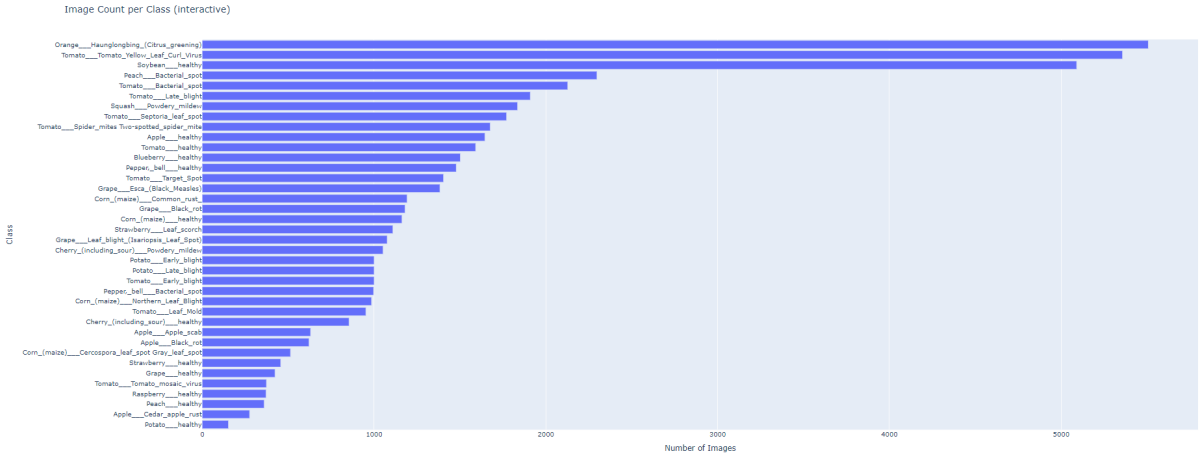


Fig. 6.2 Interactive bar chart of the image count per class (exploratory analysis output)

6.2.4 Resolution and brightness

Image size is uniform: every sampled image across all 38 classes is 256 × 256 pixels. Brightness, however, varies by crop. Tomato spans the widest range, from 10.4 to 192.9 on a 0–255 scale, while Orange sits in a dimmer and narrower band with a mean of 96.1 (Fig. 6.3). This variation is why aggressive brightness augmentation was avoided in the final pipeline.

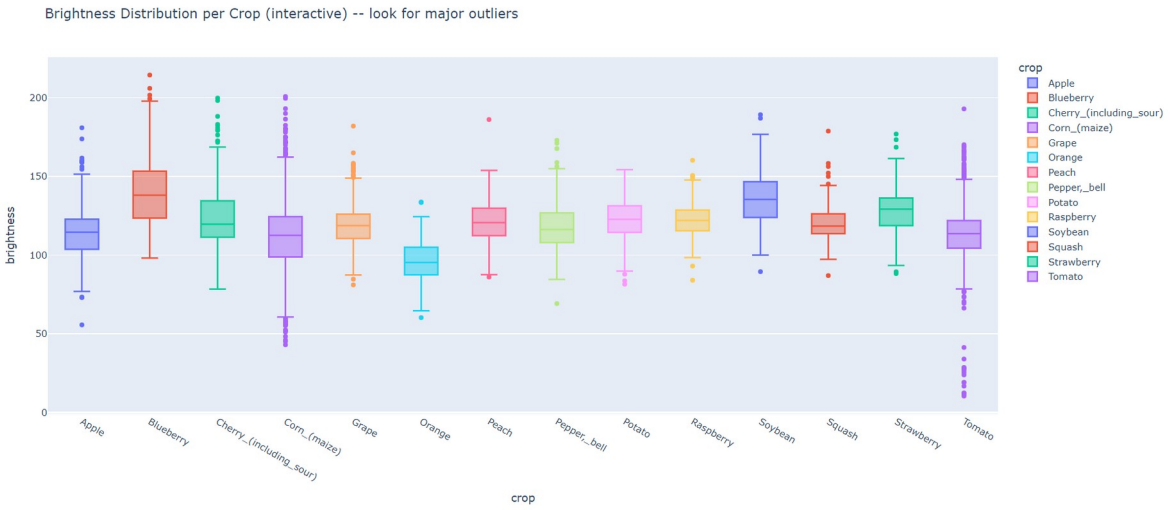


Fig. 6.3 Brightness distribution per crop species, shown as box plots (exploratory analysis output)

6.3 Data Preprocessing

The preprocessing pipeline loads each image, converts it to RGB where necessary, resizes it to the model input resolution (224 × 224 pixels for the relevant model workflow, from source images of 256 × 256 pixels), normalises the pixel values using the same scaling in training and in inference, and groups images into batches. Using identical preprocessing at training and prediction time ensures that the model sees inputs in the format it was trained on. The dataset is split 70:15:15 into training, validation and test sets with random seed 42.

Several decisions were informed directly by the exploratory analysis and by experiments recorded in Review I:

- **Class weighting** was tried to offset the class imbalance but was found empirically to reduce accuracy, so the final pipeline relies instead on full fine-tuning and the overall scale of the dataset.
- **Heavy augmentation** was trialled and dropped after it reduced accuracy.
- **Brightness augmentation** was kept mild because brightness varies by crop and aggressive changes were judged unsuitable.
- Two defects were fixed during development: a RAM and shuffle-buffer crash in the input pipeline, and a value-range bug in a RandomBrightness layer.

In practical terms the input pipeline works in the following order:

1. Read the image file from the class folder of the training, validation or test split.
2. Decode the file and convert it to three colour channels.
3. Resize it to the model input size.
4. Scale the pixel values in the way the pretrained network expects.

5. Shuffle the training images and group them into batches that are fed to the network.

Only the training images are shuffled. The validation and test images are kept in a fixed order so that results can be repeated.

6.4 Feature Engineering / Feature Extraction

The project does not use hand-designed features. Features are learned by the convolutional layers of the pretrained backbones. Lower layers capture general visual patterns such as edges, colour blobs and textures, and deeper layers capture patterns that are more specific to the task, such as lesion shape and the arrangement of spots. Because the backbones were pretrained on ImageNet, these general features are available at the start of training, and fine-tuning adapts the whole network to the leaf-disease domain. A final classification layer with 38 output units produces a probability for each class.

6.5 Model Architecture / Algorithm Design

The three candidate backbones were fine-tuned end to end at a low learning rate on the PlantVillage dataset. MobileNetV2 and DenseNet121 were trained on the full 38-class dataset using the same pipeline. EfficientNetB0 was evaluated only in a preliminary run on a reduced subset. Table 6.4 records the configuration details that are available in the supplied material.

Table 6.4 Model configuration details recorded in the project documents

Item	Recorded detail	Remark
Initialisation	ImageNet-pretrained weights for all three backbones	Review I
Training regime	End-to-end fine-tuning at a low learning rate	Numerical learning rate not stated in the supplied reports
MobileNetV2 data	Full 38-class dataset	Same pipeline as DenseNet121
DenseNet121 data	Full 38-class dataset	Same pipeline as MobileNetV2
EfficientNetB0 data	Reduced 22-class subset, 150 images per class, 128-pixel input	Preliminary; not directly comparable
Training hardware	NVIDIA T4 GPU on Google Colab	Review II, Section 3.6
Output layer	38 classes with a class-index mapping saved with the model	Review II, Section 3.1

Fine-tuning means that the weights learned on ImageNet are not thrown away but are adjusted slowly using the leaf images. A low learning rate is used for this reason. If the rate were high, the early updates could wipe out the useful features that the network brought with it. Here the whole network was fine-tuned, so the early layers could also adapt a little to the colours and textures typical of leaves.

6.5.1 MobileNetV2

MobileNetV2 is a lightweight architecture that loads quickly and gives fast predictions even on a CPU-only server, which suits a web application that must respond to each uploaded image within seconds. Review II identifies it as the final classifier of the PlantRx application, with a reported test accuracy of about 97 percent on the 38-class PlantVillage test set.

6.5.2 DenseNet121

DenseNet121 uses dense connectivity, in which each layer reuses the feature maps of all preceding layers. Review I argued that this improves gradient flow and helps distinguish the fine-grained lesion texture, colour and pattern differences among classes that share similar leaf shapes. Because deployment is single-image, on-demand inference rather than real-time video, the additional parameters were considered an acceptable cost. On the full 38-class dataset it reached a test accuracy of about 94 percent.

6.5.3 EfficientNetB0

EfficientNetB0 was evaluated in a preliminary run on a reduced 22-class subset with 150 images per class and 128-pixel inputs. The accuracy obtained, approximately 85.76 percent on a 660-image test set, is reported for completeness only. It is not a like-for-like comparison with the full 38-class experiments.

6.6 System / Model Implementation

6.6.1 Model preparation

After fine-tuning, the selected model was saved together with its trained weights and the class-index mapping. Saving the model as a single file lets the web server load it once at start-up instead of retraining or rebuilding the network for every request. The class-index mapping records which of the 38 output neurons corresponds to which crop–disease label, for example Tomato — Early blight or Apple — healthy, so that a numeric prediction can be converted back into a readable name.

6.6.2 Inference pipeline

Table 6.5 Inference pipeline for each uploaded image (Review II)

Step	Stage	Description
1	Validation	The server checks that the file is a supported image type and can be opened without errors
2	Preprocessing	The image is converted to RGB, resized to the model input resolution and normalised as in training
3	Prediction	A batch of one image is passed through the network, which outputs a probability for each of the 38 classes
4	Decoding	The class with the highest probability is selected as the predicted disease and its probability is the confidence score
5	Remedy lookup	The predicted class label is used as a key to fetch remedy and prevention guidance
6	Response	Disease name, confidence, remedy and prevention text are returned to the browser and displayed

Table 6.6 Content returned by the inference service for each analysed image

Field	Content (Review II)
Disease name	Predicted crop and disease, or healthy if no disease is detected
Confidence score	Probability of the predicted class, so the user can judge reliability
Top candidate predictions	The three most likely classes with their probabilities
Remedy and prevention text	Observed symptoms, guidance plan with immediate actions and treatment options, and preventive practices
History entry (signed-in users)	Image, predicted disease, confidence, remedy and date and time

6.6.3 Remedy database and guidance

The remedy database maps each of the 38 classes to a disease description, symptoms, recommended treatment and preventive measures. In the result screen the stored content appears as observed symptoms, a recommended guidance plan with immediate actions and treatment options, and preventive practices. Presenting the remedy on the same page as the diagnosis is intended to shorten the interval between symptom onset and treatment.

6.6.4 Web application

The back end is written in Python with FastAPI and served with Uvicorn. It receives the uploaded image through a REST API, runs the inference pipeline, queries the remedy database and returns the result as JSON, with Pydantic validating request and response data. Configuration values such as secret keys are kept in environment variables in a .env file rather

than in source code. The front end is written in JavaScript using React with Vite. The application is packaged with Docker through a Dockerfile and a docker-compose.yml so that the back end and front end can be started together on any machine.

6.6.5 AI Assistant

The AI Assistant is described in Review II as an FAQ chatbot. User messages are matched against a prepared set of questions and answers and the closest match is returned. If no suitable match is found, the chatbot asks the user to rephrase the question. Typical questions concern the cause and spread of a disease, how and when to apply a treatment, how to prevent recurrence and how to photograph a leaf.

6.6.6 Deployment

PlantRx AI is delivered as a web application that opens in the browser of a desktop, laptop or smartphone. The user does not install anything. Because each request carries a single image and the answer is needed on demand, the server runs comfortably on a CPU alone and no GPU is needed at deployment time.

The back end and the front end are packaged with Docker, using a Dockerfile and a docker-compose.yml file. This means both parts start in the same way on any machine, which avoids the usual problems of different library versions. Secret keys and other settings are read from a .env file, so they never appear in the source code. The model is loaded once when the server starts, which keeps each prediction fast.

6.7 Validation Checks

The system checks inputs at several points. The server verifies the file type and that the file can be opened before inference (Table 6.5, step 1). The upload screen limits a submission to five images of up to 10 MB each in JPG, PNG or WEBP format and shows a short message about image framing (Fig. 6.6). The home page describes a confidence check that flags results below 60 percent so that treatment is not applied on a doubtful prediction (Fig. 9.1). The result page also states that severity and affected-area percentage are not provided by the classification model (Fig. 6.7).

6.8 User Interface Implementation

The figures below show the main screens of PlantRx AI. Figure 6.4 is the home page, Fig. 6.5 is the upload screen, Fig. 6.6 collects the plant details, and Figs. 6.7 and 6.8 show a diagnosis result.

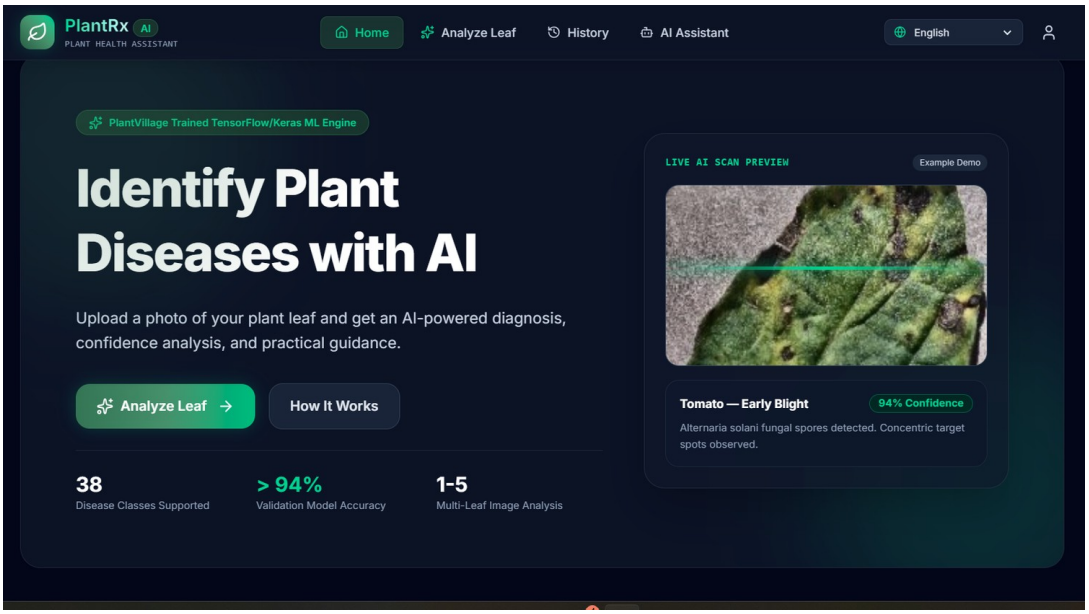


Fig. 6.4 PlantRx AI home page

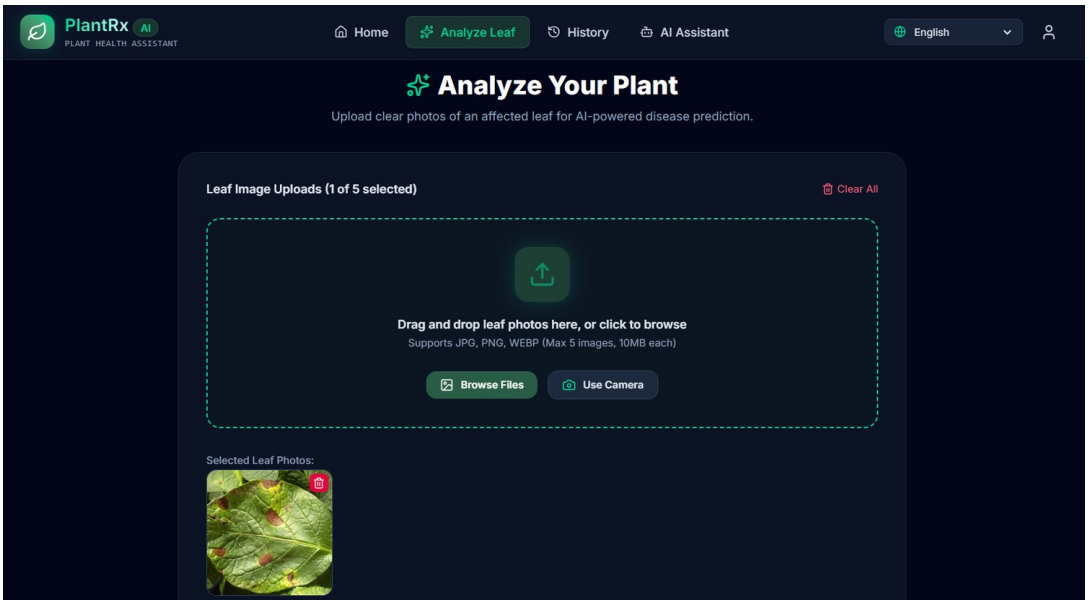


Fig. 6.5 Analyze Your Plant page for uploading up to five leaf images (JPG, PNG or WEBP, 10 MB each)

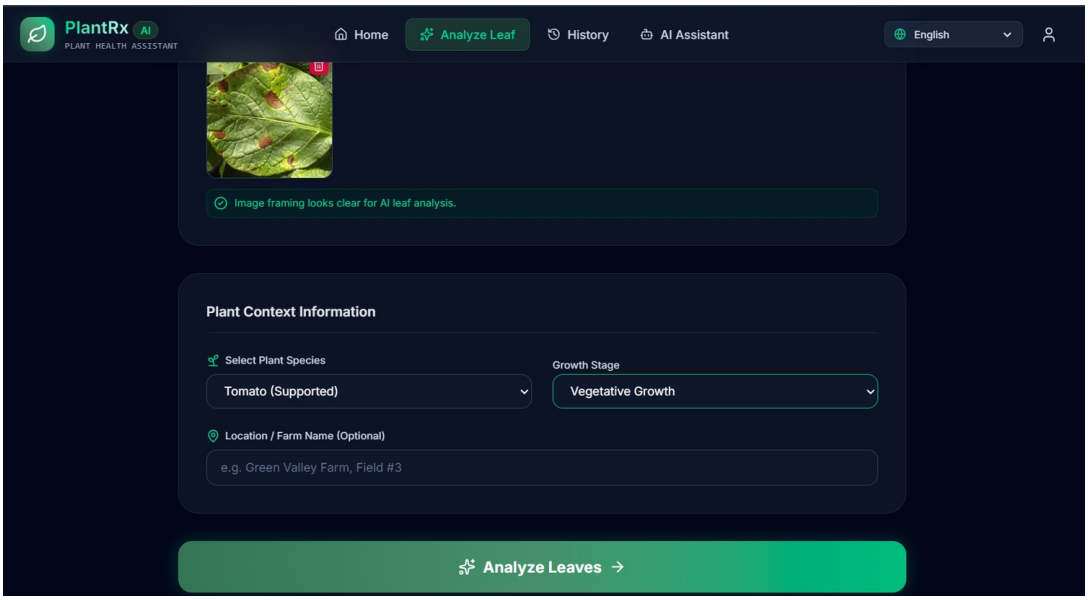


Fig. 6.6 Plant context information: species, growth stage and optional location

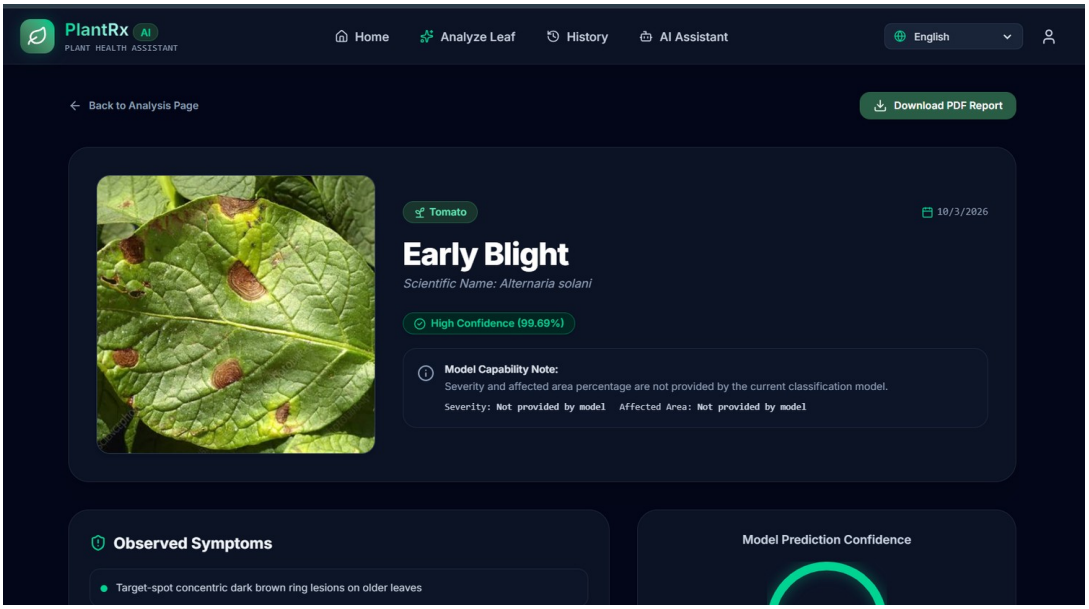


Fig. 6.7 Result page header: crop, predicted disease, confidence badge and model capability note

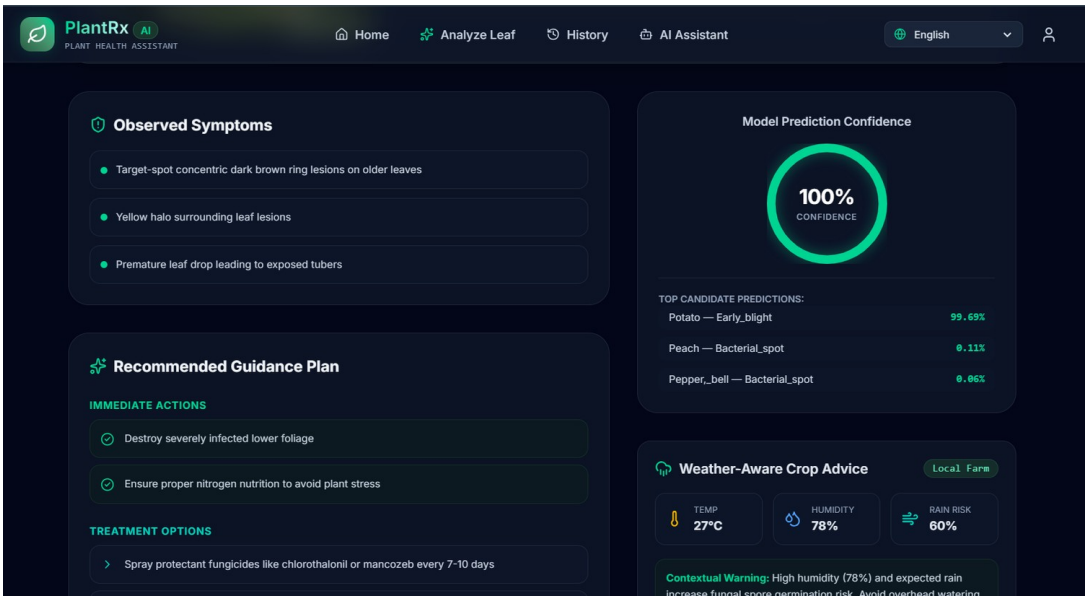


Fig. 6.8 Result page detail: observed symptoms, guidance plan, prediction confidence, top candidates and weather-aware advice

The weather panel on the result page (Fig. 6.8) shows a temperature of 27 °C, humidity of 78 percent and a rain risk of 60 percent for a location labelled “Local Farm”, along with a warning that high humidity and expected rain increase the risk of fungal spread.

CHAPTER 7 TESTING AND EVALUATION

7.1 Test Cases

Table 7.1 lists the functional test cases for the application. Each case follows from a functional requirement in Table 4.1.

Table 7.1 Functional test cases

ID	Test case	Expected result
TC1	Upload one valid JPG leaf image	Preview is shown and the image is accepted
TC2	Upload more than five images	Selection is limited to five images
TC3	Upload an unsupported or corrupt file	File is rejected with a message
TC4	Upload an image larger than 10 MB	File is rejected
TC5	Select plant species and growth stage	Selection is accepted and sent with the request
TC6	Analyse a valid image	Disease, confidence and top three classes are shown
TC7	Check guidance for the predicted class	Symptoms, actions, treatment and prevention are shown
TC8	Low-confidence prediction	Result is flagged when confidence is below 60 percent
TC9	Ask the assistant an FAQ question	Closest answer is returned, or a request to rephrase
TC10	Sign in and open history	Only the user’s own earlier diagnoses are listed
TC11	Download the PDF report	A PDF of the result is produced
TC12	Change the interface language	Interface text changes to the chosen language

7.2 Unit Testing

Unit tests for this system would address the individual functions of the inference pipeline: file-type and readability validation, conversion to RGB, resizing to the model input resolution, normalisation, decoding of the highest-probability class through the class-index mapping, and the remedy lookup for each of the 38 labels. A particularly useful unit check is that every class label produced by the model has a matching entry in the remedy database, because a missing key would produce an empty guidance panel.

- File validation accepts the supported image types and rejects unreadable files.
- Preprocessing returns an array of the model input shape with normalised values.
- The decoded label for each of the 38 output indices matches the class-index mapping.
- Every label has a corresponding remedy entry, including the healthy-leaf classes.
- The response object contains disease name, confidence, top three candidates and guidance fields.

7.3 Integration Testing

Integration testing verifies that the components work together: the front end sends an upload to the REST API, the back end validates and preprocesses it, the model returns probabilities, the remedy database returns guidance for the predicted class, and the response is rendered and, for a signed-in user, written to the history. The environment variables and the Docker configuration should also be exercised by starting both services together from the compose file on a clean machine.

7.4 System Testing

System testing exercises the complete user journey of Section 5.1 in a browser on both a desktop and a mobile device. The supplied screenshots show the upload, context, result and guidance screens in a desktop browser, which indicates that these screens rendered at the time of capture. They do not by themselves demonstrate correct behaviour across all inputs.

A complete system test follows the user from start to finish. The tester opens the site, chooses a language, uploads one to five leaf photographs, fills in the plant details, runs the analysis and reads the result. The tester then opens the guidance, the weather advice and the assistant, downloads the PDF report, and checks that the diagnosis appears in the history. The same steps should be repeated on a phone browser, with a poor-quality photograph, and with a leaf of a plant that is not in the dataset, to see how the system behaves when the input is not ideal.

7.5 Model Evaluation / Performance Evaluation

Each model was evaluated on the held-out test set and compared in Review I. Table 7.2 gives the results.

Table 7.2 Model results

Model	Experimental setting	Test accuracy
MobileNetV2	Full 38-class dataset; fine-tuned end to end at a low learning rate	About 97% (best validation accuracy 97.53%)
DenseNet121	Full 38-class dataset; same pipeline as MobileNetV2	About 94%
EfficientNetB0	Reduced 22-class subset, 150 images per class, 128-pixel input	About 85.76% on a 660-image test set

The EfficientNetB0 run used a smaller subset and a lower image size, so it cannot be compared directly with the other two models.

7.5.1 Choosing the final model

Accuracy was not the only point considered when choosing the final model. The application answers one image at a time, so loading time and prediction time matter as much as a small gain in accuracy. MobileNetV2 is the lightest of the three networks. It loads quickly, predicts in a few seconds on a CPU-only server and needs less memory, which keeps hosting simple and cheap.

DenseNet121 has many more parameters, and its dense connections make it heavier to serve. EfficientNetB0 was tested only on a small subset, so there was not enough evidence to prefer it. On the full dataset MobileNetV2 scored about 97 percent against about 94 percent for DenseNet121, so it was both the more accurate and the lighter option, and it was selected.

7.6 Performance Metrics

Accuracy is the proportion of test images classified correctly. For a 38-class problem with strong class imbalance, accuracy alone can hide poor performance on small classes, so per-class precision, recall and F1-score are the standard complementary measures. Precision for a class is the fraction of images predicted as that class that truly belong to it; recall is the fraction of images of that class that were found; and the F1-score is their harmonic mean. A confusion matrix shows which classes are mistaken for which.

Table 7.3 Standard evaluation metrics for the 38-class classifier

Metric	Definition	Why it matters here
Accuracy	Correct predictions divided by all predictions on the test set	Overall measure; can hide weak small classes under class imbalance
Precision (per class)	Images correctly predicted as a class divided by all images predicted as that class	Shows how often a positive diagnosis is trustworthy
Recall (per class)	Images correctly predicted as a class divided by all images truly in that class	Shows how many cases of a disease are missed
F1-score	Harmonic mean of precision and recall	Single balanced figure per class
Confusion matrix	Counts of true class against predicted class	Reveals which similar diseases are confused

7.7 Comparative Analysis

The three backbones serve different purposes. MobileNetV2 is small and fast, DenseNet121 reuses features across layers at a higher computing cost, and EfficientNetB0 was tried only on a reduced set. MobileNetV2 reached about 97 percent on the full dataset, ahead of DenseNet121 at about 94 percent. Since the application answers one image at a time on a CPU-only server, the fast and light MobileNetV2 was chosen as the final model.

CHAPTER 8 RESULTS AND CONCLUSION

8.1 Results

The principal results of the project are the following. First, a profile of the PlantVillage cleaned dataset was produced showing a strongly imbalanced class distribution, uniform 256 × 256 image size, and crop-dependent brightness. Second, three pretrained backbones were fine-tuned and compared; MobileNetV2 reached about 97 percent test accuracy on the full 38-class task, and a best validation accuracy of 97.53 percent is reported in the source logs. Third, a browser-based application was built that connects the trained classifier to a remedy database and presents the diagnosis, confidence, top candidates and guidance in one view.

Table 8.1 Objectives and outcomes

Objective (Section 1.3)	Outcome
Dataset profiling	Class balance, image size and brightness analysed (Figs. 6.1–6.3, Table 6.3)
Preprocessing pipeline	Loading, resizing, normalising and batching pipeline built (Section 6.3)
Backbone comparison and selection	Three backbones compared; MobileNetV2 selected with about 97% accuracy (Table 7.2)
Remedy database	Guidance for all 38 classes, shown on the result page (Fig. 6.8)
Web application	PlantRx AI built and its screens captured (Figs. 6.4–6.8)
Supporting features	Assistant, history, PDF export and language selection added to the application

8.2 Discussion / Analysis

The exploratory analysis shaped the design in a concrete way. The strong class imbalance suggested class weighting, but the experiment showed that it reduced accuracy, so it was dropped. The wide brightness spread for some crops and the narrow, darker band for others made aggressive brightness augmentation unattractive. The decision to serve a lightweight backbone follows from the deployment context: a single-image, on-demand request on a CPU-only server benefits more from fast loading and prediction than from a marginal difference in accuracy.

MobileNetV2 reached about 97 percent test accuracy on the full dataset and suits the intended deployment well. It loads quickly and answers within seconds even on a CPU-only server. The remaining weakness is that the report gives accuracy only; per-class precision, recall and F1-score would show how the model behaves on the small classes.

The interface results are also worth noting. The result page puts the diagnosis, the confidence, the symptoms and the treatment steps on one screen, and adds weather advice on top. This turns a plain class label into something a grower can act on. The page also says openly that it does not give disease severity, which keeps the user from reading more into the result than the model can support.

8.3 Limitations

- The classifier is restricted to the 38 classes of the dataset; other crops and diseases cannot be identified.
- Held-out test accuracy on a dataset of this kind may not carry over to photographs taken in the field with cluttered backgrounds, varied lighting or multiple leaves.
- The dataset is heavily imbalanced and no per-class precision, recall or F1-score is available in the supplied reports.
- Severity and affected-area percentage are not provided by the classification model, as stated on the result page.
- The assistant is FAQ-based and answers only from a prepared set of questions.
- Results are reported as accuracy only; per-class precision, recall and F1-score are not included.

8.4 Lessons Learnt

- Transfer learning from ImageNet-pretrained backbones converges faster and works well for a dataset of this size.
- Interventions that look sensible in theory, such as class weighting and heavy augmentation, must be tested; here they reduced accuracy.
- Exploratory analysis of resolution and brightness is worth doing before fixing the preprocessing pipeline.
- Memory behaviour of the input pipeline, for example the shuffle buffer, can crash a training run on a hosted notebook.
- Reports written at different review stages can drift apart; keeping a single record of final model, metrics and artefacts avoids conflicting statements.
- Interface text such as an accuracy badge should be tied to a named model and a named split so that it can be checked.

8.5 Conclusion

PlantRx shows that a lightweight, transfer-learning model can turn a plain leaf photograph into a useful diagnosis. Trained on 54,284 PlantVillage images across 38 classes, MobileNetV2

reached about 97 percent test accuracy and was chosen as the final model because it is fast and runs on a CPU-only server.

The model is part of a complete web application. A user uploads up to five leaf images, adds the plant details, and gets the disease name, a confidence score, the top three candidates and clear guidance on symptoms, treatment and prevention. Because the result and the advice appear together, a farmer does not have to wait for an expert before acting.

The project is limited to the 38 classes in the dataset and reports accuracy only. Testing on field photographs, adding severity estimation and widening language support are the natural next steps. Even so, the work delivers what it set out to do: a simple, quick and practical tool for early plant disease detection.

8.6 Future Enhancements

- Resolve the DenseNet121 and EfficientNetB0 comparisons by re-running them under identical conditions on the full 38-class dataset, and report per-class precision, recall, F1-score and a confusion matrix.
- Evaluate the model on photographs taken in real field conditions and add such images to training if needed.
- Add severity and affected-area estimation, for example through segmentation, to complement classification.
- Use a live weather source for the weather advice panel.
- Extend multilingual support so that guidance is available in regional languages.
- Extend the assistant beyond FAQ matching, for example by retrieval over the remedy database, with clear separation between model predictions and generated text.
- Add a documented database schema, automated unit and integration tests and monitoring for the deployed service.

CHAPTER 9 APPENDICES

Appendix 1 – Plagiarism Certificate

The plagiarism certificate is given on the sixth page of the preliminary pages of this report.

Appendix 2 – Screenshots

The main screens of the application are shown in Figs. 6.4 to 6.8. Fig. 9.1 shows the home-page section that explains how the system works. Figs. 9.2 to 9.4 show more exploratory charts: the number of images per crop, the image widths, and width against height. All sampled images are 256 × 256 pixels.

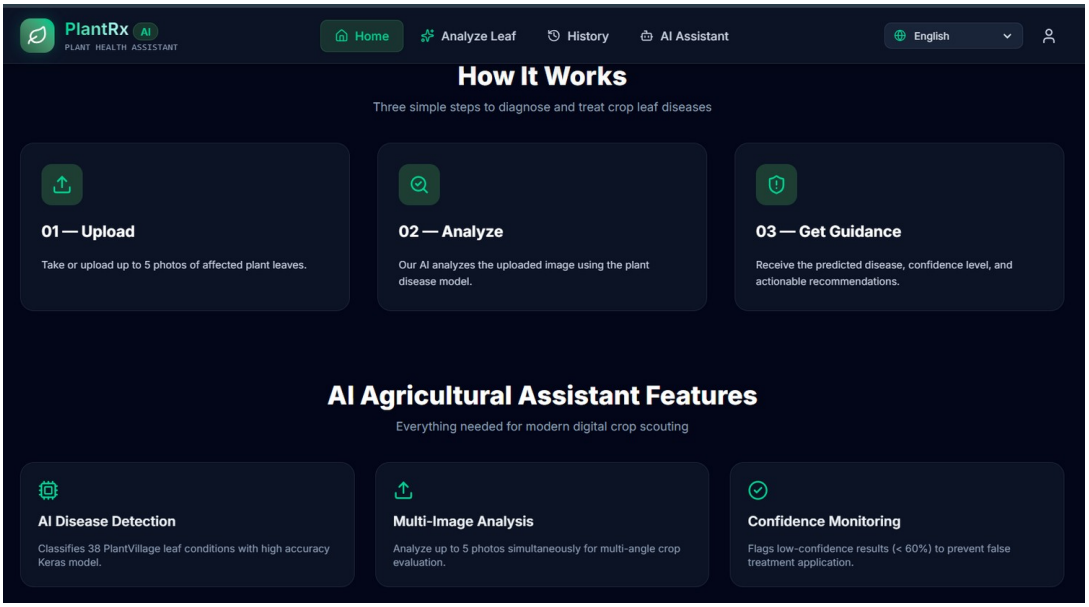


Fig. 9.1 How It Works section and AI Agricultural Assistant feature cards

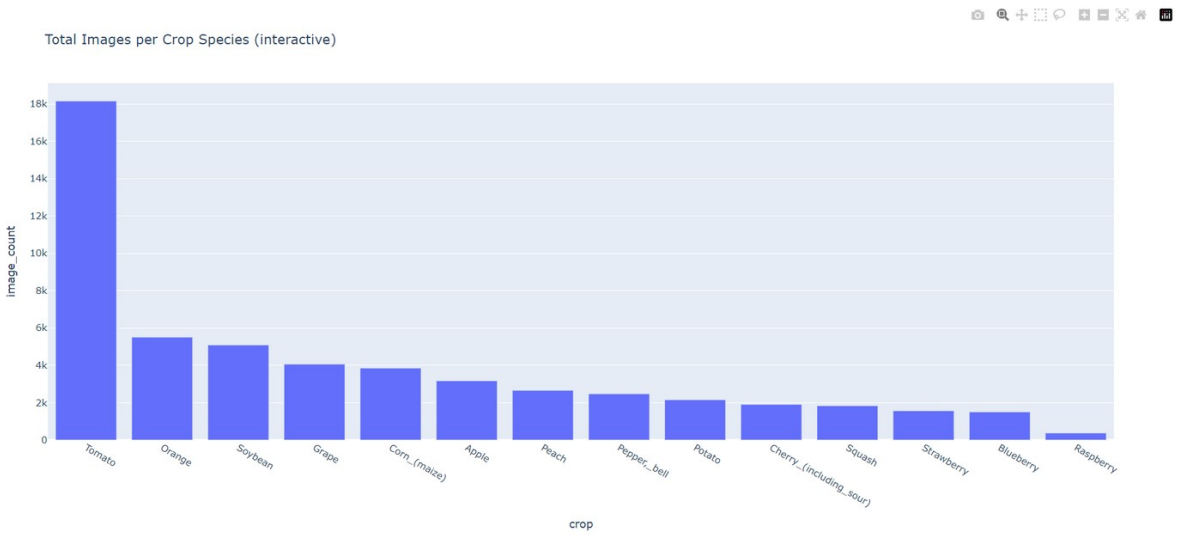


Fig. 9.2 Total images per crop species (interactive chart)

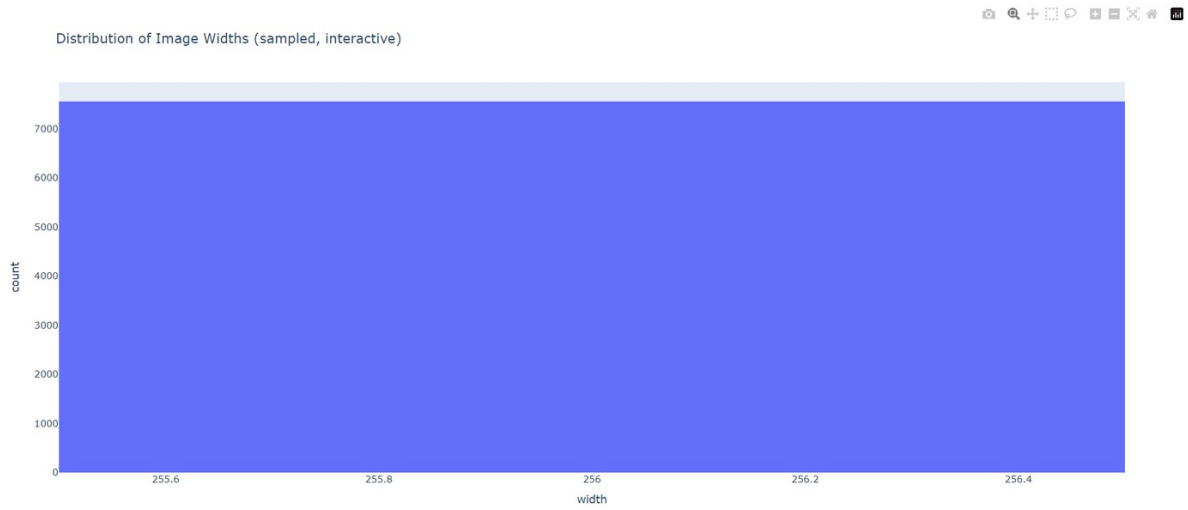


Fig. 9.3 Distribution of image widths in the sampled images

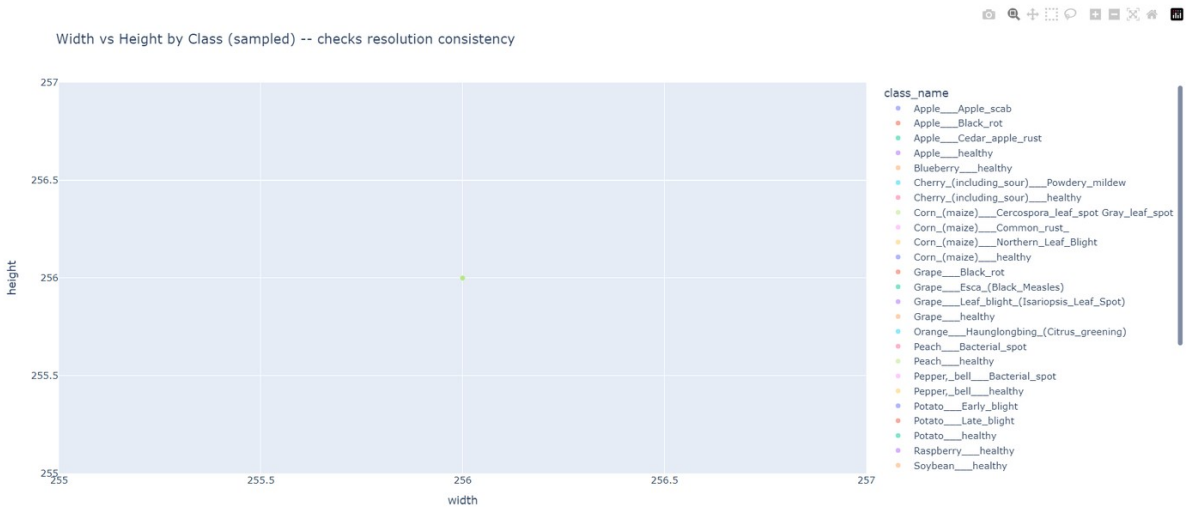


Fig. 9.4 Image width against height by class in the sampled images

Appendix 3 – Sample Coding

The code for data exploration, preprocessing and model training is kept in three Google Colab notebooks, listed below.

- Notebook 1: https://colab.research.google.com/drive/1bTow8_48Dr7s87oFB2AEh-h6sFuqYnbr?usp=sharing
- Notebook 2: https://colab.research.google.com/drive/18l0qXsAcECPq0pcjmjiqC5k-wKJ6ZGS2?usp=sharing
- Notebook 3: https://colab.research.google.com/drive/1968rdg9AD-X_3CrSQ3XU0MyaKKLOq4tm?usp=sharing

Appendix 4 – User Documentation

i) Installation Instructions

Review II states that the application is packaged with Docker through a Dockerfile and a docker-compose.yml so that the back end and front end start together. The general procedure is as follows.

6. Install Docker and Docker Compose on the host machine.
7. Obtain the project repository containing the Dockerfile and docker-compose.yml.
8. Create the .env file with the required secret keys and settings; these are not stored in the source code.
9. Place the saved model file and remedy database in the locations expected by the back end.
10. Start the services from the compose file and open the application in a browser.

ii) README / Instructions for Using the System

1. Open PlantRx AI in a browser and choose the preferred language.
2. Select Analyze Leaf. Drag and drop up to five clear leaf photographs, browse for files or use the camera.
3. Select the plant species and growth stage, and optionally enter a location or farm name.
4. Press Analyze Leaves.
5. Read the predicted disease, the confidence and the top candidate predictions. Treat results with low confidence with caution.
6. Review the observed symptoms, the guidance plan and the weather advice. Use Download PDF Report if a record is needed.
7. Use the AI Assistant for follow-up questions, and the History page to review earlier results when signed in.

iii) Tips for taking a good leaf photograph

The quality of the photograph strongly affects the result. The following simple habits help the model give a reliable answer:

- Photograph a single leaf that clearly shows the symptoms, and fill most of the frame with it.
- Use daylight and avoid strong shadows or glare on the leaf surface.
- Hold the camera steady so that the spots and lesions are sharp and not blurred.
- Keep the background simple. Place the leaf against soil or a plain surface if possible.

- Take photographs of two or three affected leaves from slightly different angles and upload them together.
- Select the correct plant species and growth stage before pressing Analyze Leaves.

Appendix 5 – Glossary

Table 9.1 Glossary of terms

Term	Meaning
CNN	Convolutional Neural Network; a deep network that learns visual features through convolution layers
Transfer learning	Initialising a model with weights learned on a large source task and adapting it to a new task
Fine-tuning	Continuing training of a pretrained model, here end to end, on the target dataset
ImageNet	A large image classification dataset used to pretrain the backbones
PlantVillage	An open collection of healthy and diseased crop leaf images
Confidence	The probability the model assigns to the predicted class
EDA	Exploratory data analysis
REST API	A web interface through which the front end sends requests to the back end
FastAPI / Uvicorn	Python web framework and the server that runs it
Docker	A container platform used to package and start the application consistently

Appendix 6 – Class Labels of the Dataset

Table 9.2 lists the 38 class labels in the order of decreasing image count shown in Fig. 6.2. Twenty-six are diseased classes and twelve are healthy classes.

Table 9.2 The 38 class labels of the PlantVillage cleaned dataset

No.	Class label (as in the dataset)	Type
1	Orange___Haunglongbing_(Citrus_greening)	Diseased
2	Tomato___Tomato_Yellow_Leaf_Curl_Virus	Diseased
3	Soybean___healthy	Healthy
4	Peach___Bacterial_spot	Diseased
5	Tomato___Bacterial_spot	Diseased
6	Tomato___Late_blight	Diseased
7	Squash___Powdery_mildew	Diseased
8	Tomato___Septoria_leaf_spot	Diseased
9	Tomato___Spider_mites Two-spotted_spider_mite	Diseased

No.	Class label (as in the dataset)	Type
10	Apple___healthy	Healthy
11	Tomato___healthy	Healthy
12	Blueberry___healthy	Healthy
13	Pepper,_bell___healthy	Healthy
14	Tomato___Target_Spot	Diseased
15	Grape___Esca_(Black_Measles)	Diseased
16	Corn_(maize)___Common_rust_	Diseased
17	Grape___Black_rot	Diseased
18	Corn_(maize)___healthy	Healthy
19	Strawberry___Leaf_scorch	Diseased
20	Grape___Leaf_blight_(Isariopsis_Leaf_Spot)	Diseased
21	Cherry_(including_sour)___Powdery_mildew	Diseased
22	Potato___Early_blight	Diseased
23	Potato___Late_blight	Diseased
24	Tomato___Early_blight	Diseased
25	Pepper,_bell___Bacterial_spot	Diseased
26	Corn_(maize)___Northern_Leaf_Blight	Diseased
27	Tomato___Leaf_Mold	Diseased
28	Cherry_(including_sour)___healthy	Healthy
29	Apple___Apple_scab	Diseased
30	Apple___Black_rot	Diseased
31	Corn_(maize)___Cercospora_leaf_spot Gray_leaf_spot	Diseased
32	Strawberry___healthy	Healthy
33	Grape___healthy	Healthy
34	Tomato___Tomato_mosaic_virus	Diseased
35	Raspberry___healthy	Healthy
36	Peach___healthy	Healthy
37	Apple___Cedar_apple_rust	Diseased
38	Potato___healthy	Healthy

CHAPTER 10 BIBLIOGRAPHY / REFERENCES

References are listed in alphabetical order of author or organisation.

[1] He, Kaiming, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. “Deep Residual Learning for Image Recognition.” IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 770–778.

[2] Huang, Gao, Zhuang Liu, Laurens van der Maaten, and Kilian Q. Weinberger. “Densely Connected Convolutional Networks.” IEEE Conference on Computer Vision and Pattern Recognition, 2017, pp. 4700–4708.

[3] Hughes, David P., and Marcel Salathé. “An Open Access Repository of Images on Plant Health to Enable the Development of Mobile Disease Diagnostics.” arXiv:1511.08060, 2015.

[4] Mohanty, Sharada P., David P. Hughes, and Marcel Salathé. “Using Deep Learning for Image-Based Plant Disease Detection.” Frontiers in Plant Science, Vol. 7, 2016, Article 1419.

[5] PlantRx Project Team. Implementation Notebooks 1–3. Google Colab. Notebook 1: https://colab.research.google.com/drive/1bTow8_48Dr7s87oFB2AEh-h6sFuqYnbr?usp=sharing; Notebook 2: https://colab.research.google.com/drive/18l0qXsAcECPq0pcjmjiqC5k-wKJ6ZGS2?usp=sharing; Notebook 3: https://colab.research.google.com/drive/1968rdg9AD-X_3CrSQ3XU0MyaKKLOq4tm?usp=sharing

[6] PlantRx Project Team. Plant Disease Detection Using Convolutional Neural Networks: Review I and Review II Submission Reports. M.Sc. ADS Mini Project, Department of Computer Applications, SRM Institute of Science and Technology, 2026.

[7] PlantVillage. PlantVillage Cleaned Dataset (project copy). Google Drive, [year to be confirmed]. https://drive.google.com/file/d/1HCDvjwXtG4By8FumdjYQre1zc_IusvBe/view?usp=sharing

[8] Sandler, Mark, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. “MobileNetV2: Inverted Residuals and Linear Bottlenecks.” IEEE Conference on Computer Vision and Pattern Recognition, 2018, pp. 4510–4520.

[9] Tan, Mingxing, and Quoc V. Le. “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks.” Proceedings of the International Conference on Machine Learning, 2019, pp. 6105–6114.

[10] TensorFlow. “TensorFlow Documentation.” TensorFlow, https://www.tensorflow.org/. Accessed [date to be completed].